

Optimizacion de Red

NVIDIA Jetson AGX Orin 64GB Developer Kit

Guia Completa Paso a Paso

JetPack 6.2.2 | Ubuntu 22.04.5 LTS | CUDA 12.6 | Kernel 5.15.185-tegra

Hardware Objetivo:

CPU: ARMv8 rev 1 (v8l) 12 nucleos @ 2.2 GHz

RAM: 64 GB LPDDR5 (unificada CPU+GPU)

GPU: Ampere (2048 CUDA cores, 64 Tensor cores)

Ethernet: 10 GbE integrado

CUDA 12.6 | cuDNN 9.3 | TensorRT 10.3 | OpenCV 4.8

Marzo 2026 | v1.0

Tabla de Contenidos

Capitulo 1 Diagnostico Inicial del Sistema

- 1.1 Identificar interfaces de red
- 1.2 Medir rendimiento actual (linea base)
- 1.3 Revisar parametros del kernel actuales

Capitulo 2 Modo de Maximo Rendimiento (nvpmodel)

- 2.1 Activar MAXN y jetson_clocks
- 2.2 Verificar frecuencias CPU/GPU/EMC

Capitulo 3 Optimizacion de Parametros del Kernel (sysctl)

- 3.1 Buffers TCP/IP
- 3.2 Control de congestion BBR
- 3.3 Escalado de ventana TCP y Fast Open
- 3.4 Desactivar IPv6 (opcional)
- 3.5 Archivo de configuracion completo

Capitulo 4 Optimizacion de la Interfaz Ethernet

- 4.1 MTU y Jumbo Frames
- 4.2 Offloads de hardware (TSO, GRO, LRO)
- 4.3 Ring buffers y colas RX/TX
- 4.4 Persistencia con systemd

Capitulo 5 Optimizacion de DNS

- 5.1 Configurar Cloudflare/Quad9 como DNS
- 5.2 DNS sobre TLS con systemd-resolved
- 5.3 Cache local de DNS

Capitulo 6 Optimizacion de Descargas

- 6.1 apt: descargas paralelas
- 6.2 aria2: descargas multihilo
- 6.3 Modelos de IA: hf_transfer y aria2
- 6.4 Docker: pulls paralelos optimizados

Capitulo 7 Verificacion y Benchmarking

- 7.1 iperf3: pruebas TCP y UDP
- 7.2 Prueba de velocidad de descarga real
- 7.3 Monitoreo continuo con iftop/nload

Capitulo 8 Script de Automatizacion Completo

Capitulo 9 Reversion de Cambios (Rollback)

Apendice Referencias y Fuentes

Introduccion

Esta guia le enseñara a optimizar completamente la red de su NVIDIA Jetson AGX Orin 64GB Developer Kit. Cubrir desde el diagnostico inicial hasta la configuracion avanzada de parametros del kernel, interfaces Ethernet, DNS, y herramientas de descarga. Cada comando se explica en detalle y esta diseñado para ejecutarse directamente en su terminal.

La optimizacion de red es critica en el Jetson porque muchas tareas de IA implican descargar modelos grandes (a menudo de 5 a 70 GB), imagenes Docker pesadas, y transferir datos entre el Jetson y otros dispositivos de la red local. Los valores por defecto de Linux son conservadores y no aprovechan la capacidad real de su hardware.¹

NOTA: Todos los comandos de esta guia estan probados para Ubuntu 22.04 LTS aarch64 con Kernel 5.15.185-tegra (JetPack 6.2.2). Si usa una version diferente, algunos parametros podrian variar.

Que va a lograr con esta guia:

- Aumentar el throughput TCP hasta un 300-500% respecto a los valores por defecto
- Reducir la latencia DNS de ~50ms a menos de ~10ms
- Acelerar descargas de modelos de IA (Hugging Face, Ollama) con herramientas multihilo
- Optimizar Docker pulls de imagenes pesadas de contenedores Jetson
- Aprovechar al maximo la interfaz Ethernet 10 GbE integrada del Orin
- Automatizar toda la configuracion con un script unificado

¹ <https://oneuptime.com/blog/post/2026-03-20-optimize-linux-network-sysctl-10gbps/view>

Capitulo 1: Diagnostico Inicial del Sistema

Antes de modificar cualquier parametro, debemos conocer el estado actual de la red. Esto nos dara una linea base para comparar despues de aplicar las optimizaciones.

1.1 Identificar Interfaces de Red

El primer paso es saber que interfaces de red tiene disponibles su Jetson. El AGX Orin Developer Kit incluye un puerto Ethernet Gigabit/10GbE (eth0) y un adaptador WiFi (wlan0). Ejecute:

```
# Listar interfaces
# Listar todas las interfaces de red activas
ip link show
# Resultado esperado (ejemplo):
# 1: lo: <LOOPBACK,UP,LOWER_UP> ...
# 2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
# 3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> ...
```

Que significa cada parte:

Elemento	Significado
lo	Loopback: interfaz virtual interna (127.0.0.1). No la modifique.
eth0	Ethernet cableado. Esta es la interfaz principal a optimizar.
wlan0	WiFi inalambrico. Util como respaldo, pero mas lento que Ethernet.
UP	La interfaz esta activa y funcionando.
LOWER_UP	Hay un cable conectado (o señal WiFi establecida).

Para ver informacion detallada de su interfaz Ethernet:

```
# Detalles de la interfaz Ethernet
# Ver detalles completos de eth0
ip addr show eth0
# Ver velocidad actual del enlace Ethernet
# Requiere: sudo apt install ethtool -v
sudo ethtool eth0 | grep -i speed
# Resultado esperado: Speed: 1000Mb/s (o 10000Mb/s si tiene 10GbE)
# Ver la ruta por defecto (gateway)
ip route show default
# Resultado: default via 192.168.1.1 dev eth0 proto dhcp ...
```

NOTA: Si ve 'Speed: 100Mb/s', revise su cable Ethernet. Use cables Cat5e o Cat6 para garantizar Gigabit. Para 10GbE necesita Cat6a o superior y un switch compatible.

1.2 Medir el Rendimiento Actual (Linea Base)

Es fundamental medir la velocidad antes de optimizar. Así podrá cuantificar exactamente cuanto mejora después de cada cambio. Instale las herramientas:

```
# Instalar herramientas
# Instalar herramientas de diagnóstico de red
sudo apt update
sudo apt install -y iperf3 speedtest-cli iftop nload curl ethtool net-tools
```

Prueba rápida con speedtest-cli:

```
# Prueba de velocidad de internet
# Medir velocidad de internet actual
speedtest-cli --simple
# Resultado esperado:
# Ping: 12.345 ms
# Download: 85.67 Mbit/s
# Upload: 45.23 Mbit/s
# GUARDE estos valores para comparar después:
speedtest-cli --simple | tee ~/red-linea-base.txt
echo "Fecha: $(date)" >> ~/red-linea-base.txt
```

Prueba de red local con iperf3:

iperf3 mide el ancho de banda real entre dos dispositivos de su red. Necesita otro computador en la misma red para ejecutar el servidor.²

```
# Prueba de red local con iperf3
# EN OTRO PC DE SU RED (el servidor):
iperf3 -s
# Esto abre el puerto 5201 y espera conexiones
# EN SU JETSON (el cliente):
iperf3 -c IP_DEL_OTRO_PC -t 30 -P 4
# -c: conectar al servidor (reemplace IP DEL OTRO PC con la IP real)
# -t 30: prueba de 30 segundos
# -P 4: usar 4 conexiones paralelas
# Prueba en reversa (descarga):
iperf3 -c IP_DEL_OTRO_PC -t 30 -P 4 -R
```

CONSEJO: Si no tiene otro PC disponible, puede usar servidores públicos de iperf3: `iperf3 -c iperf.he.net -t 10`

1.3 Revisar Parametros del Kernel Actuales

Antes de cambiar los parametros del kernel, veamos los valores actuales para entender que necesita optimizacion:

```
# Diagnostico de parametros del kernel
# Ver buffers TCP actuales
echo '=== Buffers TCP ==='
svsctl net.core.rmem max net.core.wmem max
svsctl net.ipv4.tcp rmem net.ipv4.tcp wmem
echo '=== Cola de red ==='
svsctl net.core.netdev max backlog
echo '=== Algoritmo de congestion ==='
svsctl net.ipv4.tcp congestion control
echo '=== Modulos de congestion disponibles ==='
svsctl net.ipv4.tcp available congestion control
echo '=== IPv6 ==='
svsctl net.ipv6.conf.all.disable_ipv6
echo '=== File descriptors ==='
sysctl fs.file-max
```

Guarde la salida completa para referencia:

```
# Backup de parametros
# Guardar TODOS los parametros de red actuales
svsctl -a 2>/dev/null | grep -E '^net\.' > ~/svsctl-backup-$(date +%Y%m%d).txt
echo "Backup guardado en ~/svsctl-backup-$(date +%Y%m%d).txt"
```

ADVERTENCIA: Este backup es importante. Si algo sale mal despues de los cambios, puede restaurar los valores originales comparando con este archivo.

Capitulo 2: Modo de Maximo Rendimiento (nvpmodel)

El Jetson AGX Orin tiene multiples modos de energia. Para optimizacion de red, necesitamos que todos los nucleos CPU esten activos y a maxima frecuencia, ya que el procesamiento de red (checksums, encryption, compression) consume CPU.

2.1 Activar MAXN y jetson_clocks

```
# Activar modo maximo rendimiento
# Ver el modo de energia actual
sudo nvpmodel -a
# Resultado: NV Power Mode: MODE 30W ALL (o similar)
# Cambiar al modo MAXN (maximo rendimiento)
# MAXN = todos los nucleos activos, frecuencias maximas
sudo nvpmodel -m 0
# Fijar todas las frecuencias al maximo
# Esto desactiva el throttling dinamico
sudo jetson clocks
# Verificar que se aplico correctamente
sudo nvpmodel -a
# Debe mostrar: NV Power Mode: MAXN
```

ADVERTENCIA: MAXN aumenta significativamente el consumo de energia y la temperatura. Asegurese de que su Jetson tenga buena ventilacion. Si no necesita rendimiento maximo permanente, puede volver al modo anterior con 'sudo nvpmodel -m 1' (30W) o 'sudo nvpmodel -m 2' (15W).

2.2 Verificar Frecuencias CPU/GPU/EMC

```
# Verificar rendimiento del hardware
# Instalar itop (monitor avanzado para Jetson)
sudo pip3 install jetson-stats
# Reiniciar el servicio despues de instalar
sudo systemctl restart itop.service
# Ejecutar el monitor
itop
# Use las flechas para navegar entre pestañas
# Tab 1: vista general (CPU, GPU, memoria)
# Tab 5: INFO (modelo, JetPack, CUDA, etc.)
# Alternativa rapida sin itop:
sudo tegrastats --interval 1000
```



```
# Muestra uso de CPU, GPU, memoria, temperatura en tiempo real
# Presione Ctrl+C para detener
```

Valores esperados despues de activar MAXN + jetson_clocks:

Componente	Valor esperado	Como verificar
CPU (12 nucleos)	2201 MHz (todos activos)	cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_cur_freq
GPU	Frecuencia maxima (~1.3 GHz)	cat /sys/devices/gpu.0/devfreq/17000000.ga10b/cur_freq
EMC (memoria)	Frecuencia maxima	cat /sys/kernel/debug/bpmp/debug/clk/emc/rate
Fan (ventilador)	100% velocidad	cat /sys/class/hwmon/hwmon*/pwm1

Capitulo 3: Optimizacion de Parametros del Kernel (sysctl)

sysctl es la herramienta que permite modificar parametros del kernel de Linux en tiempo real. Los parametros de red controlan el tamaño de los buffers TCP, el algoritmo de congestion, el escalado de ventana, y muchos otros aspectos criticos del rendimiento de red.²

Linux viene con valores conservadores pensados para servidores pequeños con poca RAM. Su Jetson AGX Orin tiene 64 GB de RAM, lo cual permite buffers mucho mas grandes y parametros mas agresivos.³

3.1 Buffers TCP/IP

Los buffers TCP determinan cuanta informacion puede almacenarse temporalmente antes de enviarse o procesarse. Buffers mas grandes = mayor throughput, especialmente para descargas grandes y conexiones de alta latencia.

Concepto clave – Bandwidth–Delay Product (BDP):

El BDP calcula el tamaño optimo de buffer para su conexion. Formula: $BDP = \text{Ancho de banda} \times \text{Latencia (RTT)}$. Ejemplo: Para una conexion de 1 Gbps con 20ms de latencia: $BDP = 1,000,000,000 \text{ bits/s} \times 0.020\text{s} = 20,000,000 \text{ bits} = 2.5 \text{ MB}$. Esto significa que necesita al menos 2.5 MB de buffer TCP por conexion. Los valores por defecto de Linux (~200 KB) limitarian esta conexion a solo ~80 Mbps.

Parametros a modificar:

Parametro	Default	Optimizado	Que hace
net.core.rmem_max	~212 KB	128 MB	Buffer maximo de recepcion del socket
net.core.wmem_max	~212 KB	128 MB	Buffer maximo de envio del socket
net.core.rmem_default	~212 KB	1 MB	Buffer por defecto de recepcion
net.core.wmem_default	~212 KB	1 MB	Buffer por defecto de envio
net.ipv4.tcp_rmem	4K 87K 6M	4K 1M 128M	Rango min/default/max recepcion TCP
net.ipv4.tcp_wmem	4K 16K 4M	4K 1M 128M	Rango min/default/max envio TCP

3.2 Control de Congestion BBR

BBR (Bottleneck Bandwidth and Round-trip time) es un algoritmo de control de congestion desarrollado por Google. Es superior al algoritmo CUBIC por defecto, especialmente para conexiones WAN (internet). BBR mantiene alto throughput incluso con perdida de paquetes y reduce significativamente la latencia.⁴

```
# Verificar disponibilidad de BBR
# Verificar si BBR esta disponible en su kernel
sudo modprobe tcp_bbr 2>/dev/null
sysctl net.ipv4.tcp_available_congestion_control
# Debe incluir 'bbr' en la lista
# Si no aparece bbr, su kernel Tegra puede no incluirlo.
# En ese caso, use 'cubic' (el default) que sigue siendo bueno.
```

CONSEJO: El kernel Tegra 5.15.185 generalmente incluye el modulo tcp_bbr. Si 'modprobe tcp_bbr' no da error, BBR esta disponible. Si no, los demas parametros de esta guia seguiran mejorando significativamente su red.

3.3 Escalado de Ventana TCP y Fast Open

TCP Window Scaling permite ventanas de recepcion mayores a 64 KB (el limite original de TCP). Es indispensable para conexiones de alta velocidad. TCP Fast Open reduce la latencia al establecer conexiones, enviando datos en el mismo paquete SYN.⁵

3.4 Desactivar IPv6 (Opcional)

Si su red local no usa IPv6, desactivarlo puede reducir la latencia en la resolucion de nombres DNS y eliminar trafico innecesario. Sin embargo, si su ISP usa IPv6, dejelo activado.

ADVERTENCIA: Desactivar IPv6 es opcional y solo recomendado si esta seguro de que su red solo usa IPv4. En redes modernas, IPv6 puede ser beneficioso.

3.5 Archivo de Configuracion Completo

A continuacion, el archivo de configuracion completo optimizado para su Jetson AGX Orin 64GB. Este archivo se coloca en /etc/sysctl.d/ para que se aplique automaticamente al reiniciar:

```
# Crear archivo de configuracion sysctl
# Crear el archivo de configuracion optimizado
sudo nano /etc/sysctl.d/99-jetson-network.conf
```

Copie y pegue el siguiente contenido completo:

```
# Contenido de /etc/sysctl.d/99-jetson-network.conf
# =====
```

```
# OPTIMIZACION DE RED - NVIDIA Jetson AGX Orin 64GB
# Archivo: /etc/svctl.d/99-jetson-network.conf
# Fecha: Marzo 2026 | JetPack 6.2.2 | Kernel 5.15.185-tegra
# =====
# — 1. BUFFERS TCP/IP —
# Buffer maximo de recepcion y envio del socket (128 MB)
# Justificacion: 64 GB de RAM permite buffers grandes
net.core.rmem max = 134217728
net.core.wmem max = 134217728
# Buffers por defecto (1 MB)
net.core.rmem default = 1048576
net.core.wmem default = 1048576
# TCP buffer: min / default / max (en bytes)
# Auto-tuning ajustara dentro de este rango por conexion
net.ipv4.tcp_rmem = 4096 1048576 134217728
net.ipv4.tcp_wmem = 4096 1048576 134217728
# Memoria TCP a nivel sistema (paginas de 4K)
# min=256MB, pressure=1GB, max=2GB
net.ipv4.tcp_mem = 65536 262144 524288
# — 2. CONTROL DE CONGESTION —
# BBR (Google) - superior a CUBIC para descargas WAN
# Requiere adisc fq (Fair Queueing)
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
# — 3. COLA DE RECEPCION —
# Aumentar cola de paquetes para ráfagas de tráfico
net.core.netdev_max_backlog = 16384
# Conexiones maximas en cola de escucha
net.core.somaxconn = 8192
net.ipv4.tcp_max_syn_backlog = 8192
# — 4. ESCALADO Y OPTIMIZACION TCP —
# Escalado de ventana TCP (requerido para buffers grandes)
net.ipv4.tcp_window_scaling = 1
# TCP Fast Open (cliente + servidor)
net.ipv4.tcp_fastopen = 3
# SACK - recuperacion rapida de perdida de paquetes
net.ipv4.tcp_sack = 1
net.ipv4.tcp_dsack = 1
# Timestamps - mejora precision de RTT
net.ipv4.tcp_timestamps = 1
# No reiniciar slow-start despues de inactividad
net.ipv4.tcp_slow_start_after_idle = 0
# Auto-tuning de buffer de recepcion
net.ipv4.tcp_moderate_rcvbuf = 1
```

```
# Reducir bufferbloat
net.ipv4.tcp notsent lowat = 16384
# MTU probina automatico
net.ipv4.tcp mtu probina = 1
# — 5. REUTILIZACION DE CONEXIONES —
# Reducir tiempo en TIME WAIT
net.ipv4.tcp fin timeout = 15
# Reutilizar sockets en TIME WAIT
net.ipv4.tcp tw reuse = 1
# Rango de puertos locales (mas puertos disponibles)
net.ipv4.ip local port range = 1024 65535
# — 6. KEEPALIVE —
# Detectar conexiones muertas mas rapido
net.ipv4.tcp keepalive time = 300
net.ipv4.tcp keepalive intvl = 15
net.ipv4.tcp keepalive probes = 5
# — 7. SEGURIDAD BASICA —
# Proteccion contra SYN flood
net.ipv4.tcp syncookies = 1
# Desactivar enrutamiento de fuente
net.ipv4.conf.all.accept source route = 0
net.ipv4.conf.default.accept source route = 0
# Filtrado de ruta inversa
net.ipv4.conf.all.rp filter = 1
net.ipv4.conf.default.rp filter = 1
# — 8. DESACTIVAR IPv6 (OPCIONAL) —
# Descomente las siguientes lineas SOLO si no usa IPv6
# net.ipv6.conf.all.disable ipv6 = 1
# net.ipv6.conf.default.disable ipv6 = 1
# net.ipv6.conf.lo.disable ipv6 = 1
# — 9. LIMITES DEL SISTEMA —
# Descriptores de archivo (necesario para muchas conexiones)
fs.file-max = 2097152
# Memoria virtual: reducir swappiness
vm.swappiness = 10
```

Aplique los cambios inmediatamente:

```
# Aplicar configuracion
# Aplicar todos los parametros del archivo
sudo sysctl -p /etc/sysctl.d/99-jetson-network.conf
# Verificar que se aplicaron correctamente
sysctl net.core.rmem max
# Debe mostrar: net.core.rmem max = 134217728
sysctl net.ipv4.tcp_congestion_control
```

```
# Debe mostrar: net.ipv4.tcp congestion control = bbr  
# (Si BBR no esta disponible, mostrara 'cubic')
```

NOTA: Los cambios en /etc/sysctl.d/ son persistentes: se aplican automaticamente cada vez que reinicie su Jetson. No necesita ejecutar el comando sysctl -p manualmente despues de reiniciar.

² <https://cubepath.com/docs/advanced-topics/advanced-tcp-ip-tuning-with-sysctl>

³ <https://oneuptime.com/blog/post/2026-03-20-optimize-linux-network-sysctl-10gbps/view>

⁴ <https://perlod.com/tutorials/linux-kernel-network-tuning/>

⁵ <https://www.linkedin.com/pulse/practical-optimization-network-performance-via-linux-kernel-maciel-vx51f>

Capitulo 4: Optimizacion de la Interfaz Ethernet

4.1 MTU y Jumbo Frames

MTU (Maximum Transmission Unit) define el tamaño maximo de un paquete de red. El valor por defecto es 1500 bytes. En redes locales que soporten Jumbo Frames, puede aumentarlo a 9000 bytes, lo cual reduce la sobrecarga (overhead) por paquete y mejora el throughput para transferencias grandes.⁶

```
# Configurar MTU
# Ver el MTU actual de eth0
ip link show eth0 | grep mtu
# Resultado tipico: mtu 1500
# — OPCION A: Red local con Jumbo Frames —
# SOLO si su switch/router soporta Jumbo Frames:
sudo ip link set dev eth0 mtu 9000
# Verificar que funciona (ping con paquetes grandes):
ping -M do -s 8972 GATEWAY IP
# Si funciona sin errores. Jumbo Frames esta activo
# Si falla. su red no soporta Jumbo Frames
# — OPCION B: Red estandar —
# Si NO sabe si su red soporta Jumbo Frames.
# deje el MTU en 1500 (el default)
# El parametro tcp_mtu_probing=1 de sysctl lo maneja
```

ADVERTENCIA: NO cambie el MTU a 9000 si no esta seguro de que TODOS los dispositivos entre su Jetson y el destino soporten Jumbo Frames. Un MTU incorrecto causa fragmentacion de paquetes y reduce el rendimiento. En caso de duda, deje 1500.

4.2 Offloads de Hardware (TSO, GRO, LRO)

Los offloads permiten que la tarjeta de red (NIC) maneje operaciones que normalmente haria la CPU, como calcular checksums y segmentar paquetes grandes. Esto libera la CPU para otras tareas (como inferencia de IA).⁷

Offload	Nombre completo	Que hace
TSO	TCP Segmentation Offload	La NIC divide paquetes grandes en segmentos TCP, reduciendo carga CPU
GRO	Generic Receive Offload	La NIC agrupa paquetes recibidos antes de pasarlos al kernel

Offload	Nombre completo	Que hace
LRO	Large Receive Offload	Similar a GRO pero a nivel de hardware. No siempre soportado
GSO	Generic Segmentation Offload	Version por software de TSO. Funciona como respaldo

```
# Activar offloads de hardware
# Instalar ethtool si no lo tiene
sudo apt install -v ethtool
# Ver estado actual de los offloads
sudo ethtool -k eth0 | grep -E 'offload|scatter-gather|checksum'
# Activar todos los offloads disponibles
sudo ethtool -K eth0 tso on 2>/dev/null || echo 'TSO no soportado'
sudo ethtool -K eth0 gro on 2>/dev/null || echo 'GRO no soportado'
sudo ethtool -K eth0 gso on 2>/dev/null || echo 'GSO no soportado'
sudo ethtool -K eth0 lro on 2>/dev/null || echo 'LRO no soportado'
sudo ethtool -K eth0 sg on 2>/dev/null || echo 'SG no soportado'
sudo ethtool -K eth0 rx-checksumming on 2>/dev/null
sudo ethtool -K eth0 tx-checksumming on 2>/dev/null
# Verificar que se activaron
sudo ethtool -k eth0 | grep -E 'on$'
```

4.3 Ring Buffers y Colas RX/TX

Los ring buffers son memorias intermedias en la NIC que almacenan paquetes antes de que el kernel los procese. Aumentarlos reduce la perdida de paquetes en momentos de alto trafico:

```
# Configurar ring buffers
# Ver el tamaño actual de los ring buffers
sudo ethtool -a eth0
# Mostrara: Pre-set maximums v Current hardware settings
# Aumentar al maximo soportado por el hardware
# (los valores exactos dependen de su NIC)
sudo ethtool -G eth0 rx 4096 tx 4096 2>/dev/null || \
sudo ethtool -G eth0 rx 1024 tx 1024 2>/dev/null || \
echo 'Usando valores por defecto del hardware'
```

4.4 Persistencia con systemd

Los cambios de ethtool no sobreviven un reinicio. Creemos un servicio systemd para aplicarlos automaticamente:

```
# Crear script de persistencia
# Crear el script de optimizacion de la interfaz
```



```
sudo nano /usr/local/bin/jetson-net-optimize.sh
```

```
# Contenido del script jetson-net-optimize.sh
#!/bin/bash
# /usr/local/bin/jetson-net-optimize.sh
# Optimizacion de interfaz Ethernet para Jetson AGX Orin
IFACE="eth0"
# Esperar a que la interfaz este disponible
while ! ip link show $IFACE &>/dev/null: do
sleep 1
done
# Activar offloads
ethtool -K $IFACE tso on 2>/dev/null
ethtool -K $IFACE aro on 2>/dev/null
ethtool -K $IFACE aso on 2>/dev/null
ethtool -K $IFACE lro on 2>/dev/null
ethtool -K $IFACE sq on 2>/dev/null
ethtool -K $IFACE rx-checksumming on 2>/dev/null
ethtool -K $IFACE tx-checksumming on 2>/dev/null
# Aumentar ring buffers
ethtool -G $IFACE rx 4096 tx 4096 2>/dev/null || \
ethtool -G $IFACE rx 1024 tx 1024 2>/dev/null
echo "[$(date)] Network optimizations applied to $IFACE"
```

```
# Configurar servicio systemd
# Hacer ejecutable el script
sudo chmod +x /usr/local/bin/jetson-net-optimize.sh
# Crear el servicio systemd
sudo nano /etc/systemd/system/jetson-net-optimize.service
```

```
# Contenido de jetson-net-optimize.service
[Unit]
Description=Jetson Network Interface Optimization
After=network.target
Wants=network.target
[Service]
Type=oneshot
ExecStart=/usr/local/bin/jetson-net-optimize.sh
RemainAfterExit=yes
[Install]
WantedBy=multi-user.target
```

```
# Activar servicio
# Activar el servicio para que se ejecute al iniciar
```

```
sudo systemctl daemon-reload
sudo systemctl enable jetson-net-optimize.service
sudo systemctl start jetson-net-optimize.service
# Verificar que funcione
sudo systemctl status jetson-net-optimize.service
```

⁶ <https://forums.developer.nvidia.com/t/orin-unable-to-reach-10-gbps-networking/261872>

⁷ <https://oneuptime.com/blog/post/2026-03-20-optimize-linux-network-sysctl-10gbps/view>

Capitulo 5: Optimizacion de DNS

El DNS (Domain Name System) traduce nombres de dominio (como huggingface.co) a direcciones IP. Un DNS lento añade latencia a cada conexion nueva. Optimizar el DNS mejora la velocidad de todas las operaciones de red.⁸

5.1 Configurar Cloudflare/Quad9 como DNS

Los servidores DNS de su ISP suelen ser lentos. Cloudflare (1.1.1.1) y Quad9 (9.9.9.9) son mas rapidos y ofrecen seguridad adicional:

Proveedor DNS	IP Primaria	IP Secundaria	Ventaja principal
Cloudflare	1.1.1.1	1.0.0.1	Velocidad (~11ms promedio mundial)
Quad9	9.9.9.9	149.112.112.112	Seguridad (bloquea dominios maliciosos)
Google	8.8.8.8	8.8.4.4	Confiabilidad y cobertura global

5.2 DNS sobre TLS con systemd-resolved

Ubuntu 22.04 usa systemd-resolved como su resolver DNS por defecto. Vamos a configurarlo para usar DNS sobre TLS (DoT), que cifra las consultas DNS para mayor privacidad y puede evitar manipulacion por parte del ISP.⁹

```
# Configurar DNS over TLS
# Paso 1: Editar la configuracion de svstemd-resolved
sudo nano /etc/systemd/resolved.conf

# Contenido de /etc/systemd/resolved.conf
[Resolve]
# DNS primarios con DNS over TLS
DNS=1.1.1.1#cloudflare-dns.com 1.0.0.1#cloudflare-dns.com
# DNS de respaldo
FallbackDNS=9.9.9.9#dns.quad9.net 8.8.8.8#dns.google
# Activar DNS over TLS
DNSOverTLS=yes
# Activar cache DNS local
Cache=yes
# Desactivar LLMNR (no necesario en la mavoria de redes)
LLMNR=no
# Activar mDNS para .local (util en redes locales)
MulticastDNS=yes
```

```
# Anlizar y verificar DNS
# Paso 2: Reiniciar el servicio DNS
sudo systemctl restart systemd-resolved
# Paso 3: Verificar la configuracion
resolvectl status
# Debe mostrar:
# DNS Servers: 1.1.1.1 1.0.0.1
# DNS over TLS: yes
# Paso 4: Probar la resolucion DNS
resolvectl query huggingface.co
# Debe resolver a una IP rapidamente (<10ms)
# Comparar velocidad DNS:
# Antes (ISP DNS tipico):
time dig @$(ip route show default | awk '{print $3}') huggingface.co
# Despues (Cloudflare con DoT):
time resolvectl query huggingface.co
```

5.3 Cache Local de DNS

systemd-resolved ya incluye un cache DNS incorporado. Para verificar que esta funcionando y ver estadísticas:

```
# Cache DNS
# Ver estadísticas del cache DNS
resolvectl statistics
# Resultado esperado (despues de usar un rato):
# Cache:
# Current Cache Size: 42
# Cache Hits: 156
# Cache Misses: 23
# Limpiar el cache si es necesario:
resolvectl flush-caches
resolvectl statistics # Cache Size volvera a 0
```

⁸ <https://oneuptime.com/blog/post/2026-03-02-how-to-use-systemd-resolved-for-dns-configuration-on-ubuntu/view>

⁹ <https://www.adityathebe.com/systemd-resolved-dns-over-tls/>

Capitulo 6: Optimizacion de Descargas

Este capitulo se enfoca en acelerar las descargas mas comunes en un workflow de IA: paquetes del sistema (apt), modelos de Hugging Face (5-70 GB), e imagenes Docker para contenedores Jetson.

6.1 apt: Descargas Paralelas

Por defecto, apt descarga un paquete a la vez. Podemos configurarlo para descargar varios simultaneamente y reintentarlo automaticamente en caso de fallo:

```
# Optimizar apt
# Crear archivo de configuracion para apt
sudo nano /etc/apt/apt.conf.d/99-jetson-optimize

# Contenido de /etc/apt/apt.conf.d/99-jetson-optimize
// /etc/apt/apt.conf.d/99-jetson-optimize
// Descargas paralelas y optimizaciones
Acquire::Languages "none": // No descargar traducciones
Acquire::Queue-Mode "access": // Modo paralelo por servidor
Acquire::Retries "5": // Reintentar 5 veces si falla
Acquire::http::Pipeline-Depth "10": // Pipeline HTTP de 10 niveles
APT::Acquire::Retries "5"; // Reintentos adicionales

# Verificar
# Probar que funciona:
sudo apt update
# Deberia ser notablemente mas rapido que antes
```

6.2 aria2: Descargas Multihilo

aria2 es un descargador de archivos por linea de comandos que soporta descargas multihilo y multi-fuente. Es ideal para archivos grandes porque abre multiples conexiones al mismo servidor, multiplicando la velocidad de descarga.¹⁰

```
# Instalar y configurar aria2
# Instalar aria2
sudo apt install -y aria2

# Crear un archivo de configuracion global
mkdir -p ~/.config/aria2
nano ~/.config/aria2/aria2.conf

# Contenido de aria2.conf
# ~/.config/aria2/aria2.conf
```

```
# Configuración optimizada para Jetson AGX Orin 64GB
# 16 conexiones por servidor (máximo)
max-connection-per-server=16
# Dividir archivo en 16 partes
split=16
# No dividir archivos menores a 10 MB
min-split-size=10M
# 5 descargas simultáneas
max-concurrent-downloads=5
# Continuar descargas interrumpidas
continue=true
# 10 reintentos si falla
max-tries=10
# Timeout de 600 segundos
timeout=600
# No asignar espacio antes (más rápido para empezar)
file-allocation=none
```

```
# Ejemplos de uso de aria2
# Ejemplo de uso básico:
aria2c -x 16 -s 16 "https://ejemplo.com/archivo-grande.tar.gz"
# -x 16: 16 conexiones por servidor
# -s 16: dividir en 16 partes
# Con barra de progreso detallada:
aria2c -x 16 -s 16 --summary-interval=5 "URL DEL ARCHIVO"
# Descargar múltiples archivos desde una lista:
aria2c -x 16 -s 16 -i 3 -i lista-de-urls.txt
# -j 3: 3 archivos simultáneos
```

6.3 Modelos de IA: hf_transfer y aria2

Los modelos de Hugging Face pueden ser enormes (5-70 GB). Hay dos métodos optimizados para descargarlos rápidamente:¹¹

Método A: hf_transfer (Recomendado)

hf_transfer es una extensión en Rust para huggingface_hub que puede alcanzar velocidades superiores a 1 GB/s en conexiones rápidas:¹²

```
# Descargar modelos con hf_transfer
# Instalar hf_transfer
pip3 install -U 'huggingface_hub[cli]' hf_xet
# Configurar variables de entorno para máximo rendimiento
# Añadir a su ~/.bashrc:
```

```
echo 'export HF_XET_HIGH_PERFORMANCE=1' >> ~/.bashrc
echo 'export HF_XET_NUM_CONCURRENT_RANGE_GETS=16' >> ~/.bashrc
source ~/.bashrc

# Descargar un modelo (ejemplo: Owen 2.5 7B)
huggingface-cli download Owen/Owen2.5-7B-Instruct \
--local-dir ~/modelos/qwen2.5-7b
```

Metodo B: aria2 para Hugging Face

```
# Descargar modelos con aria2

# Paso 1: Clonar el repo sin los archivos grandes
GIT_LFS_SKIP_SMUDGE=1 git clone \
https://huggingface.co/Owen/Owen2.5-7B-Instruct
cd Owen2.5-7B-Instruct

# Paso 2: Generar lista de archivos para aria2
git lfs ls-files -n | xargs -d '\n' -I {} \
echo -e 'https://huggingface.co/Owen/Owen2.5-7B-Instruct/resolve/main/{}\n out={}' \
>> files.txt

# Paso 3: Descargar con aria2 (8 archivos simultaneos, 16 conexiones c/u)
aria2c -i 8 -x 16 -s 16 -i files.txt

# Si necesita autenticacion:
huggingface-cli login
aria2c -i 8 -x 16 -s 16 -i files.txt \
--header="Authorization: Bearer $(cat ~/.cache/huggingface/token)"
```

6.4 Docker: Pulls Paralelos Optimizados

Las imagenes Docker para Jetson (como las de jetson-containers) pueden ser de 5-20 GB. Por defecto, Docker descarga solo 3 capas simultaneamente. Vamos a optimizar esto:¹³

```
# Optimizar Docker

# Editar o crear la configuracion de Docker
sudo nano /etc/docker/daemon.json
```

```
# Contenido de /etc/docker/daemon.json
{
  "storage-driver": "overlay2",
  "max-concurrent-downloads": 10,
  "max-concurrent-uploads": 10,
  "log-driver": "local",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  },
}
```

```
"dns": ["1.1.1.1", "8.8.8.8"],
"default-ulimits": {
  "nofile": {
    "Name": "nofile",
    "Hard": 65535,
    "Soft": 65535
  },
  "live-restore": true,
  "runtimes": {
    "nvidia": {
      "path": "nvidia-container-runtime",
      "runtimeArgs": []
    },
    "default-runtime": "nvidia"
  }
}
```

Aplicar cambios de Docker

```
# Aplicar los cambios
sudo systemctl restart docker
# Verificar la configuración
docker info | grep -E 'Concurrent|Storage|Runtime'
# Debe mostrar:
# Max Concurrent Downloads: 10
# Storage Driver: overlay2
# Default Runtime: nvidia
```

CONSEJO: El campo 'default-runtime': 'nvidia' es importante para su Jetson. Permite que los contenedores accedan a la GPU sin necesidad de especificar --runtime nvidia cada vez.

¹⁰ <https://nixsanctuary.com/how-to-manage-and-speed-up-your-large-file-downloads-with-aria2/>

¹¹ <https://dev.to/susumuota/faster-and-more-reliable-hugging-face-downloads-using-aria2-and-gnu-parallel-4f2b>

¹² <https://discuss.huggingface.co/t/download-speed-way-too-slow/169824>

¹³ <https://oneuptime.com/blog/post/2026-01-16-docker-daemon-tuning/view>

Capitulo 7: Verificacion y Benchmarking

Despues de aplicar todas las optimizaciones, es momento de medir los resultados y compararlos con la linea base del Capitulo 1.

7.1 iperf3: Pruebas TCP y UDP

```
# Bateria de pruebas iperf3
# — Prueba TCP basica (upload) —
iperf3 -c IP SERVIDOR -t 30
# — Prueba TCP con multiples streams —
iperf3 -c IP SERVIDOR -t 30 -P 4
# — Prueba TCP reversa (download) —
iperf3 -c IP SERVIDOR -t 30 -R
# — Prueba bidireccional —
iperf3 -c IP SERVIDOR -t 30 --bidir
# — Prueba UDP (mide jitter v perdida de paquetes) —
iperf3 -c IP SERVIDOR -t 30 -u -b 500M
# -u: modo UDP
# -b 500M: ancho de banda objetivo de 500 Mbps
# — Prueba con buffer grande —
iperf3 -c IP SERVIDOR -t 30 -w 8M
# -w 8M: buffer de 8 MB (aprovecha nuestros sysctl)
```

Valores de referencia para el Jetson AGX Orin:

Prueba	Antes (tipico)	Despues (esperado)	Mejora
TCP single stream (1 Gbps LAN)	400-600 Mbps	900-950 Mbps	~60-130%
TCP 4 streams (1 Gbps LAN)	700-850 Mbps	940-960 Mbps	~15-35%
TCP single (10 GbE LAN)	2-4 Gbps	8-9.5 Gbps	~150-350%
UDP jitter	0.5-2 ms	0.05-0.2 ms	~90%
Speedtest internet	Variable	Maximo de su plan ISP	Variable

7.2 Prueba de Velocidad de Descarga Real

```
# Pruebas de velocidad de descarga

# Prueba 1: speedtest-cli
speedtest-cli --simple | tee ~/red-despues-optimizacion.txt

# Prueba 2: Descarga de un archivo de prueba con aria2
# (archivo de 100 MB del servidor de pruebas de Hetzner)
aria2c -x 16 -s 16 --summary-interval=5 \
'https://speed.hetzner.de/100MB.bin' \
-d /tmp -o test-100mb.bin
rm -f /tmp/test-100mb.bin

# Prueba 3: Descarga con wget (single thread) para comparar
wget -O /dev/null 'https://speed.hetzner.de/100MB.bin' 2>&1 | tail -2

# Comparar línea base vs optimizado:
echo '=== ANTES ==='
cat ~/red-línea-base.txt
echo '=== DESPUES ==='
cat ~/red-despues-optimizacion.txt
```

7.3 Monitoreo Continuo con iftop/nload

```
# Herramientas de monitoreo en tiempo real

# iftop: monitor de trafico por conexion (tipo 'top' para la red)
sudo iftop -i eth0

# Muestra: fuente, destino, velocidad en tiempo real
# Presione 'd' para salir

# nload: monitor de ancho de banda por interfaz
nload eth0

# Muestra: grafico ASCII de entrada/salida
# Presione 'd' para salir

# Alternativa rapida sin instalar nada:
watch -n 1 'cat /proc/net/dev | grep eth0'
```

Capitulo 8: Script de Automatizacion Completo

Este script unificado aplica TODAS las optimizaciones de esta guia en un solo paso. Puede ejecutarlo en una instalacion fresca de JetPack 6.2.2:

```
# Crear el script maestro
# Descargar y ejecutar el script
# (O cree el archivo manualmente con nano)
nano ~/jetson-net-setup.sh

# Contenido de jetson-net-setup.sh
#!/bin/bash
# =====
# jetson-net-setup.sh
# Optimizacion completa de red para Jetson AGX Orin 64GB
# JetPack 6.2.2 | Ubuntu 22.04 | Kernel 5.15.185-tegra
# =====
set -euo pipefail
GREEN='\033[0:32m'
YELLOW='\033[1:33m'
RED='\033[0:31m'
NC='\033[0m'

log() { echo -e "${GREEN}[OK]${NC} $1": }
warn() { echo -e "${YELLOW}[!!]${NC} $1": }
error() { echo -e "${RED}[ERROR]${NC} $1": }

echo "====="
echo " Jetson AGX Orin - Network Optimization"
echo " $(date)"
echo "====="

# — Verificar que somos root —
if [ "$EUID" -ne 0 ]; then
error "Ejecute con sudo: sudo bash $0"
exit 1
fi

# — Paso 1: Backup —
echo ""
echo ">>> Paso 1/7: Creando backup de configuracion actual..."
BACKUP_DIR="/root/net-backup-$(date +%Y%m%d %H%M%S)"
mkdir -p "$BACKUP_DIR"
svsctl -a 2>/dev/null | grep -E '^net\.' > "$BACKUP_DIR/svsvctl-original.txt"
cp /etc/sysctl.conf "$BACKUP_DIR/" 2>/dev/null || true
```

```
cd /etc/docker/daemon.json "$BACKUP DIR/" 2>/dev/null || true
cd /etc/systemd/resolved.conf "$BACKUP DIR/" 2>/dev/null || true
log "Backup guardado en $BACKUP DIR"

# — Paso 2: Instalar herramientas —
echo ""
echo ">>> Paso 2/7: Instalando herramientas de red..."
apt update -qq
apt install -y -qq aria2 iperf3 ethtool iftop nload \
speedtest-cli net-tools curl 2>/dev/null
log "Herramientas instaladas"

# — Paso 3: Modo MAXN —
echo ""
echo ">>> Paso 3/7: Activando modo MAXN..."
nvmmodel -m 0 2>/dev/null && log "nvmmodel: MAXN" || warn "nvmmodel no disponible"
jetson_clocks 2>/dev/null && log "jetson_clocks activado" || warn "jetson_clocks no disponible"

# — Paso 4: sysctl —
echo ""
echo ">>> Paso 4/7: Aplicando optimizaciones del kernel..."
cat > /etc/sysctl.d/99-jetson-network.conf << 'SYSCTL EOF'
# BUFFERS TCP/IP
net.core.rmem max = 134217728
net.core.wmem max = 134217728
net.core.rmem default = 1048576
net.core.wmem default = 1048576
net.ipv4.tcp_rmem = 4096 1048576 134217728
net.ipv4.tcp_wmem = 4096 1048576 134217728
net.ipv4.tcp_mem = 65536 262144 524288
# CONGESTION CONTROL
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
# COLAS
net.core.netdev_max_backlog = 16384
net.core.somaxconn = 8192
net.ipv4.tcp_max_syn_backlog = 8192
# TCP OPTIMIZATION
net.ipv4.tcp_window_scaling = 1
net.ipv4.tcp_fastopen = 3
net.ipv4.tcp_sack = 1
net.ipv4.tcp_dsack = 1
net.ipv4.tcp_timestamps = 1
net.ipv4.tcp_slow_start_after_idle = 0
net.ipv4.tcp_moderate_rcvbuf = 1
net.ipv4.tcp_notsent_lowat = 16384
```

```
net.ipv4.tcp mtu probing = 1
# REUTILIZACION
net.ipv4.tcp fin timeout = 15
net.ipv4.tcp tw reuse = 1
net.ipv4.ip local port range = 1024 65535
# KEEPALIVE
net.ipv4.tcp keepalive time = 300
net.ipv4.tcp keepalive intvl = 15
net.ipv4.tcp keepalive probes = 5
# SEGURIDAD
net.ipv4.tcp syncookies = 1
net.ipv4.conf.all.accept source route = 0
net.ipv4.conf.default.accept source route = 0
net.ipv4.conf.all.rp filter = 1
net.ipv4.conf.default.rp filter = 1
# SISTEMA
fs.file-max = 2097152
vm.swappiness = 10
SYSCTL EOF
modprobe tcp bbr 2>/dev/null || true
svsctl -p /etc/svsvctl.d/99-ietson-network.conf 2>/dev/null
loa "Parametros del kernel aplicados"

# — Paso 5: Ethernet offloads —
echo ""
echo ">>> Paso 5/7: Optimizando interfaz Ethernet..."
IFACE="eth0"
if ip link show $IFACE &>/dev/null: then
ethtool -K $IFACE tso on 2>/dev/null || true
ethtool -K $IFACE aro on 2>/dev/null || true
ethtool -K $IFACE gso on 2>/dev/null || true
ethtool -K $IFACE lro on 2>/dev/null || true
ethtool -K $IFACE sg on 2>/dev/null || true
ethtool -G $IFACE rx 4096 tx 4096 2>/dev/null || \
ethtool -G $IFACE rx 1024 tx 1024 2>/dev/null || true
loa "Offloads v ring buffers configurados para $IFACE"
else
warn "Interfaz $IFACE no encontrada"
fi

# Crear servicio de persistencia
cat > /usr/local/bin/ietson-net-optimize.sh << 'NET EOF'
#!/bin/bash
IFACE="eth0"
while ! ip link show $IFACE &>/dev/null: do sleep 1; done
ethtool -K $IFACE tso on gro on gso on lro on sg on 2>/dev/null
```

```

ethtool -G $IFACE rx 4096 tx 4096 2>/dev/null || \
ethtool -G $IFACE rx 1024 tx 1024 2>/dev/null
NET EOF
chmod +x /usr/local/bin/jetson-net-optimize.sh
cat > /etc/systemd/system/jetson-net-optimize.service << 'SVC EOF'
[Unit]
Description=Jetson Network Interface Optimization
After=network.target
[Service]
Type=oneshot
ExecStart=/usr/local/bin/jetson-net-optimize.sh
RemainAfterExit=yes
[Install]
WantedBy=multi-user.target
SVC EOF
systemctl daemon-reload
systemctl enable jetson-net-optimize.service 2>/dev/null
log "Servicio de persistencia creado y activado"

# — Paso 6: DNS —
echo ""
echo ">>> Paso 6/7: Configurando DNS optimizado..."
cat > /etc/systemd/resolved.conf << 'DNS EOF'
[Resolve]
DNS=1.1.1.1#cloudflare-dns.com 1.0.0.1#cloudflare-dns.com
FallbackDNS=9.9.9.9#dns.quad9.net 8.8.8.8#dns.google
DNSOverTLS=yes
Cache=yes
LLMNR=no
MulticastDNS=yes
DNS EOF
systemctl restart systemd-resolved
log "DNS configurado (Cloudflare + Quad9 con DoT)"

# — Paso 7: Docker y APT —
echo ""
echo ">>> Paso 7/7: Optimizando Docker y APT..."
# APT
cat > /etc/apt/apt.conf.d/99-jetson-optimize << 'APT EOF'
Acquire::Languages "none";
Acquire::Queue-Mode "access";
Acquire::Retries "5";
Acquire::http::Pipeline-Depth "10";
APT EOF
log "APT optimizado"

# Docker (preservar configuracion existente si tiene nvidia runtime)

```

```

if command -v docker &>/dev/null: then
if [ ! -f /etc/docker/daemon.json ]: then
cat > /etc/docker/daemon.json << 'DOCKER EOF'
{
  "storage-driver": "overlay2",
  "max-concurrent-downloads": 10,
  "max-concurrent-uploads": 10,
  "log-driver": "local",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  },
  "dns": ["1.1.1.1", "8.8.8.8"],
  "default-ulimits": {
    "nofile": { "Name": "nofile", "Hard": 65535, "Soft": 65535 }
  },
  "live-restore": true,
  "runtimes": {
    "nvidia": {
      "path": "nvidia-container-runtime",
      "runtimeArgs": []
    }
  },
  "default-runtime": "nvidia"
}
DOCKER EOF
systemctl restart docker
log "Docker optimizado"
else
warn "daemon.json ya existe. Aduene manualmente:"
warn ' "max-concurrent-downloads": 10'
fi
else
warn "Docker no instalado. Omitiendo."
fi
# — Resumen —
echo ""
echo "=====
echo " OPTIMIZACION COMPLETADA"
echo "=====
echo " Backup: $BACKUP DIR"
echo " svscctl: /etc/svscctl.d/99-jetson-network.conf"
echo " DNS: /etc/svscctl.d/resolved.conf"
echo " APT: /etc/apt/apt.conf.d/99-jetson-optimize"

```

```
echo " ETH: /etc/systemd/system/jetson-net-optimize.service"
echo ""
echo " Ejecute estas pruebas para verificar:"
echo " speedtest-cli --simple"
echo " systemctl net.ipv4.tcp congestion control"
echo " resolvectl status | head -10"
echo "====="
```

```
# Ejecutar el script
```

```
# Hacer ejecutable y ejecutar
chmod +x ~/jetson-net-setup.sh
sudo bash ~/jetson-net-setup.sh
```


Capitulo 9: Reversion de Cambios (Rollback)

Si experimenta problemas despues de las optimizaciones, puede revertir todos los cambios. El script de automatizacion crea un backup automatico.

```
# Revertir TODAS las optimizaciones
# — Revertir svscctl —
sudo rm /etc/svscctl.d/99-ietson-network.conf
sudo svscctl --svsystem # Recarga la configuracion por defecto
# — Revertir DNS —
# Restaurar el archivo original desde el backup:
sudo cp /root/net-backup-*/resolved.conf /etc/svsystemd/resolved.conf
sudo svsystemctl restart svsystemd-resolved
# — Revertir APT —
sudo rm /etc/apt/apt.conf.d/99-ietson-optimize
# — Revertir Ethernet service —
sudo svsystemctl disable ietson-net-optimize.service
sudo rm /etc/svsystemd/svsystem/ietson-net-optimize.service
sudo rm /usr/local/bin/ietson-net-optimize.sh
sudo svsystemctl daemon-reload
# — Revertir Docker (si modifico daemon.ison) —
sudo cp /root/net-backup-*/daemon.ison /etc/docker/daemon.ison
sudo svsystemctl restart docker
# — Reiniciar para aplicar todo —
sudo reboot
```

CONSEJO: Despues de revertir y reiniciar, su sistema volvera a los valores originales. Ejecute las pruebas del Capitulo 1 para confirmar que todo esta como antes.

Apendice: Referencias y Fuentes

1. OneUptime — Optimize Linux Network Stack with sysctl for 10Gbps Networks
<https://oneuptime.com/blog/post/2026-03-20-optimize-linux-network-sysctl-10gbps/view>
2. CubePath — Advanced TCP/IP Tuning with Sysctl
<https://cubepath.com/docs/advanced-topics/advanced-tcp-ip-tuning-with-sysctl>
3. PerLod Hosting — Best Linux Kernel Network Tuning in 2025
<https://perlod.com/tutorials/linux-kernel-network-tuning/>
4. LinkedIn - Maciel — Practical Optimization of Network Performance via Linux Kernel
<https://www.linkedin.com/pulse/practical-optimization-network-performance-via-linux-kernel-maciel-vx51f>
5. NVIDIA Developer Forums — Orin Unable to Reach 10 Gbps Networking
<https://forums.developer.nvidia.com/t/orin-unable-to-reach-10-gbps-networking/261872>
6. OneUptime — Benchmark Network Performance with iperf3
<https://oneuptime.com/blog/post/2026-03-02-benchmark-network-performance-iperf3-ubuntu/view>
7. OneUptime — systemd-resolved for DNS Configuration on Ubuntu
<https://oneuptime.com/blog/post/2026-03-02-how-to-use-systemd-resolved-for-dns-configuration-on-ubuntu/view>
8. Aditya Thebe — Setup DNS Over TLS with systemd-resolved
<https://www.adityathebe.com/systemd-resolved-dns-over-tls/>
9. DEV Community - Susumuota — Faster Hugging Face Downloads Using aria2 and GNU Parallel
<https://dev.to/susumuota/faster-and-more-reliable-hugging-face-downloads-using-aria2-and-gnu-parallel-4f2b>
10. Hugging Face Forums — Download speed optimization discussion
<https://discuss.huggingface.co/t/download-speed-way-too-slow/169824>
11. Nix Sanctuary — Manage and Speed-up Large File Downloads with Aria2
<https://nixsanctuary.com/how-to-manage-and-speed-up-your-large-file-downloads-with-aria2/>
12. OneUptime — Tune Docker Daemon for High-Performance Workloads
<https://oneuptime.com/blog/post/2026-01-16-docker-daemon-tuning/view>
13. LinuxConfig — Test Network Speed on Linux CLI with iperf3
<https://linuxconfig.org/how-to-test-network-ethernet-speed-on-linux-cli-with-iperf3>
14. Arm Learning Paths — Network Performance Testing with iperf3
<https://learn.arm.com/learning-paths/servers-and-cloud-computing/microbenchmark-network-iperf3/setup/>